

บทที่ 5

เอสเอสแอล ,โอเพนเอสเอสแอล และ ระบบการรับรองสิทธิ์ผู้ใช้ (SSL,OpenSSL and Certificate Authority (CA))

1. SSL/TLS

SSL โพรโตคอลเป็นโพรโตคอลที่ได้รับการออกแบบมา เพื่อทำให้เกิดการสื่อสารอย่างปลอดภัยขึ้น โดยโพรโตคอลสแต็กของ SSL จะอยู่ตรงกลางระหว่างชั้นแอปพลิเคชันเลเยอร์ (Application Layer) กับชั้นทรานสปอร์ตเลเยอร์(Transport Layer) นั่นคือ ชั้น SSL Layer จะเป็นชั้นที่เอาไว้ใช้สำหรับการสื่อสารที่ปลอดภัย ข้อดีคือเราสามารถพัฒนาแอปพลิเคชันได้อย่างเต็มที่ เนื่องจาก การทำให้เกิดความปลอดภัยจะเป็นภาระหน้าที่ของชั้น SSL โดย SSL จะอาศัยที่ซีพีโพรโตคอลในการรับส่งข้อมูล เพราะการส่งข้อมูลของ SSL ข้อมูลที่ถูกส่งไปนั้นเป็นข้อมูลที่ถูกเข้ารหัส ถ้าเกิดข้อมูลส่วนใดสูญหายขึ้นมาในระหว่างการส่งข้อมูลจะทำให้การถอดรหัสข้อมูลจะได้ข้อมูลที่ไม่ถูกต้องที่ซีพีจะเป็นตัวช่วยรับประกันว่าข้อมูลที่ถูกส่งไปผู้รับจะต้องได้รับอย่างครบถ้วน ดังนั้น SSL สามารถที่พัฒนาโพรโตคอลที่จะทำให้เกิดความปลอดภัยที่สูงขึ้นได้โดยไม่ต้องไปกังวลกับความถูกต้องในการส่งข้อมูล

1.1 บทบาทของ SSL (SSL Role)

SSL โพรโตคอลจะประกอบไปด้วยเซตของข้อความ (Message) และบทบาท (Role) ต่าง ๆ ที่เกี่ยวข้องเมื่อมีการส่งหรือรับข้อมูลซึ่งกันและกัน

SSL จะถูกกำหนดให้มีสองบทบาทสำหรับขบวนการติดต่อสื่อสารกันระหว่างสองระบบ โดยบทบาทของระบบแรกจะต้องมีบทบาทเป็นไคลเอนต์ และอีกระบบจะต้องมีบทบาทเป็นเซิร์ฟเวอร์ การแยกความแตกต่างนี้มีความสำคัญเพราะว่า SSL จะปฏิบัติต่อบทบาททั้งสองนี้อย่างแตกต่างกันโดยสิ้นเชิง ไคลเอนต์จะเป็นผู้เริ่มต้นการติดต่อสื่อสารอย่างปลอดภัยขึ้นมา จากนั้นเซิร์ฟเวอร์จะเป็นผู้ตอบสนองต่อความต้องการของไคลเอนต์ ภาพที่เห็นได้ชัดของบทบาททั้งสองนี้คือ เว็บเบราว์เซอร์ (Web Browser) จะมีบทบาทเป็นไคลเอนต์ เว็บเซิร์ฟเวอร์(Web Server) จะมีบทบาทเป็นเซิร์ฟเวอร์ นั่นคือเมื่อนำ SSL ไปใช้กับแอปพลิเคชันใด ๆ จะต้องคำนึงถึงความแตกต่างของระบบทั้งสองให้ชัดเจน

เหตุที่ SSL ต้องแยกความแตกต่างนี้เนื่องจากกระบวนการของ SSL จะต้องมีการทำการต่อรองค่าพารามิเตอร์ต่าง ๆ ในการสร้างการติดต่อสื่อสารที่ปลอดภัยระหว่างกัน

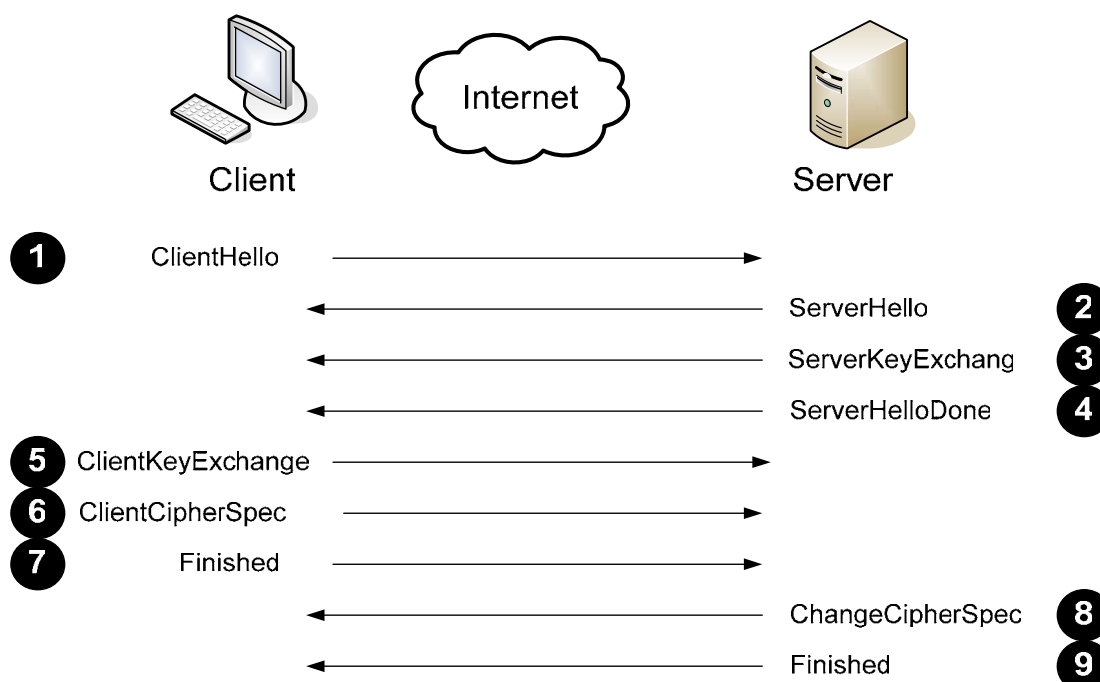
ไคลเอ็นต์จะเป็นผู้เริ่มการติดต่อสื่อสาร ซึ่งเป็นคำแนะนำค่าพารามิเตอร์ต่าง ๆ ที่ตนเองสามารถรองรับได้ส่งไปให้แก่เซิร์ฟเวอร์ และเซิร์ฟเวอร์จะเป็นผู้ทำการตัดสินใจขั้นสุดท้ายว่าทั้งสองระบบนี้จะใช้พารามิเตอร์ตัวใดในการสร้างการติดต่อสื่อสารที่ปลอดภัย ถึงแม้ว่าการตัดสินใจขั้นสุดท้ายจะอยู่ที่ฝั่งเซิร์ฟเวอร์ แต่มีข้อจำกัดอยู่ว่าเซิร์ฟเวอร์จะสามารถเลือกค่าพารามิเตอร์ต่างได้จากค่าพารามิเตอร์ที่ไคลเอ็นต์ได้ส่งมาให้แล้วเท่านั้น

1.2 ข้อความของ SSL (SSL Message)

เมื่อ SSL ไคลเอ็นต์และเซิร์ฟเวอร์ทำการติดต่อสื่อสารกัน กระบวนการภายในการติดต่อสื่อสารกันนั้นจะเป็นการส่งข้อความของ SSL (SSL Message) ระหว่างกัน ซึ่งข้อความดังกล่าวจะมีความหมายที่แตกต่างกันไปตามเฟสในการสร้างการติดต่อสื่อสารที่ปลอดภัยโดยใช้ SSL โพรโตคอล เช่น Alert เป็นข้อความที่ทำให้ทราบว่ามีการติดต่อสื่อสารล้มเหลวหรือเกิดการผิดปกติความปลอดภัยของ SSL, Application Data เป็นข้อมูลจริง(Plaintext) ที่ต้องการทำการส่งให้กันซึ่งมันจะต้องถูกทำการเข้ารหัส การระบุตัวตนของผู้ส่งหรือรับและตรวจสอบความถูกต้องของข้อมูลโดยใช้กระบวนการของ SSL โพรโตคอล ฯลฯ

1.3 การสร้างการเชื่อมต่อแบบ SSL โพรโตคอลโดยใช้การเข้ารหัสข้อมูล

กระบวนการพื้นฐานในการสร้างการเชื่อมต่อที่ปลอดภัยโดยใช้ SSL โพรโตคอล คือการใช้การเข้ารหัสข้อมูลโดยไคลเอ็นต์จะรับกุญแจสาธารณะ(Public key) ที่เซิร์ฟเวอร์ได้ส่งมาให้แล้วทำการสร้างกุญแจลับ(Secret key) นำกุญแจลับที่ได้มาเข้ารหัสด้วยกุญแจสาธารณะที่ได้รับมา จากนั้นส่งกลับไปให้แก่เซิร์ฟเวอร์และใช้กุญแจลับนี้ในการเข้ารหัสข้อมูลที่จะทำการส่งและถอดรหัสข้อมูลที่ได้รับ ซึ่งกระบวนการดังกล่าวจะเป็นการส่งข้อความของ SSL ตามเฟสต่าง ๆ ดังรูป 5-1 ซึ่งสามารถอธิบายได้ดังนี้



รูปที่ 5-1 จะใช้ ขบวนการสร้างการเชื่อมต่อโดย SSL แบบเข้ารหัสอย่างเดียว

1. ClientHello

ClientHello จะเป็นข้อความที่ไคลเอนต์ใช้สำหรับการเริ่มต้นการสื่อสารด้วย SSL โพรโตคอลโดยข้อความนี้จะเป็นการนำเสนอค่าพารามิเตอร์ต่าง ๆ ที่ตัวเองสามารถรองรับได้ส่งไปให้แก่เซิร์ฟเวอร์ โดยองค์ประกอบที่สำคัญของ ClientHello มีดังนี้

- **Version** จะเป็นตัวบอกให้ทราบว่าเวอร์ชันสูงสุดของ SSL โพรโตคอลที่ไคลเอนต์สามารถรองรับได้ ซึ่ง SSL เวอร์ชันสาม เป็นเวอร์ชันที่นิยมนำไปใช้กันอย่างกว้างขวางในระบบอินเทอร์เน็ต แต่ในมุมมองของเซิร์ฟเวอร์จะมองว่าไคลเอนต์สามารถรองรับได้ทุกเวอร์ชัน เช่น ไคลเอนต์ส่ง ClientHello มาโดยระบุเป็น SSL เวอร์ชันสามแต่เซิร์ฟเวอร์สามารถรองรับได้สูงสุดเพียงเวอร์ชันสอง เซิร์ฟเวอร์จะเลือกใช้เวอร์ชันสอง ในกรณีนี้ไคลเอนต์สามารถที่จะทำการติดต่อสื่อสารกับเซิร์ฟเวอร์ต่อไปโดยใช้ SSL เวอร์ชันสองหรือจะยกเลิกการติดต่อสื่อสารกับเซิร์ฟเวอร์ไปเลยก็ได้
- **RandomNumber** ตัวเลขสุ่มโดยฟิลด์นี้จะมีค่า 32 ไบต์โดย 4 ไบต์แรกจะเป็นค่าของวันและเวลาเหตุการณ์ที่ต้องมีการระบุวันเวลาเพราะว่าเพื่อเป็นการสร้างความมั่นใจว่าในการติดต่อสื่อสารหนึ่งครั้งจะไม่มีไคลเอนต์คนใดที่จะใช้ตัวเลขสุ่มตัวเดียวกันถึงสองครั้ง นั่นคือมันสามารถที่จะแก้ไขปัญหาของการโจมตีแบบ

แอ็กชันรีเพลย์ (Action Replay) ได้ โดยค่าที่เหลืออีก 28 ไบต์จะเป็นตัวเลขสุ่มซึ่งจำเป็นต้องสร้างมาจากฟังก์ชันที่สามารถสร้างตัวเลขสุ่มได้อย่างปลอดภัย

- **SessionID** ในการติดต่อสื่อสารระหว่างไคลเอ็นต์และเซิร์ฟเวอร์จะเรียกสั้น ๆ ว่า เซสชัน (Session) ถ้าเซิร์ฟเวอร์ทำการเก็บเซสชันของไคลเอ็นต์เอาไว้เมื่อไคลเอ็นต์ทำการติดต่อเข้ามาใหม่เซิร์ฟเวอร์จะสามารถตรวจสอบได้จากเซสชันที่ได้เก็บไว้ทำให้การติดต่อสื่อสารมีความรวดเร็วขึ้นแต่การใช้เซสชันใน SSL นั้นมีความยุ่งยากและซับซ้อน โดยปกติแล้วฟิลด์นี้จะไม่มีความหมายใดๆ เมื่อเราใช้การติดต่อ SSL แบบปกติ
- **CipherSuite** จะเป็นฟิลด์ที่ไคลเอ็นต์จะทำการใส่รหัสของอัลกอริทึมในการเข้ารหัสถอดรหัสที่ไคลเอ็นต์สามารถรองรับได้

2. ServerHello

เมื่อเซิร์ฟเวอร์ได้รับข้อความ ClientHello เซิร์ฟเวอร์จะทำการตอบกลับด้วย ServerHello ไคลเอ็นต์จะนำเสนอค่าพารามิเตอร์ต่างๆ ให้แก่เซิร์ฟเวอร์ เซิร์ฟเวอร์จะทำการเลือกค่าพารามิเตอร์เหล่านั้นแล้วส่งกลับไปด้วยข้อความ ServerHello โดยฟิลด์ของ ServerHello จะเป็นดังนี้

- **Version** จะเป็นฟิลด์ที่ระบุการติดต่อสื่อสารระหว่างไคลเอ็นต์และเซิร์ฟเวอร์จะใช้ SSL เวอร์ชัน
- **RandomNumber** เซิร์ฟเวอร์จะรับตัวเลขสุ่มจากไคลเอ็นต์และนำค่าตัวเลขนั้นมาสุ่มอีกครั้งด้วยฟังก์ชันเดียวกับไคลเอ็นต์ ซึ่งไคลเอ็นต์จะใช้ตัวเลขสุ่มนี้ในการสร้างกุญแจลับ ซึ่งจะเป็นเซสชันคีย์ที่ใช้ในเซสชันของไคลเอ็นต์และเซิร์ฟเวอร์
- **SessionID** จะไม่ใช่ค่าเดียวกันกับ SessionID ของไคลเอ็นต์ ซึ่งค่านี้จะเป็นค่าที่ใช้ในการระบุถึงไคลเอ็นต์ใดๆ ที่เชื่อมต่อเข้ามายังเซิร์ฟเวอร์
- **CipherSuite** ในฝั่งของไคลเอ็นต์ค่านี้จะอยู่ในรูปพหุพจน์ เซิร์ฟเวอร์จะทำการเลือกค่าและค่าที่ได้จะอยู่ในรูปของเอกพจน์

3. ServerKeyExchange

เมื่อเซิร์ฟเวอร์ทำการส่งข้อความ ServerHello ไปแล้ว เฟสต่อไปเซิร์ฟเวอร์จะทำการส่งข้อความ ServerKeyExchange ไปให้แก่ฝั่งไคลเอนต์ ในขณะที่ฟิลด์ CipherSuite จะเป็นตัวที่บอกถึงชนิดของอัลกอริทึมและขนาดของคีย์ข้อความ ServerKeyExchange จะเป็นค่าของกุญแจสาธารณะของเซิร์ฟเวอร์ ที่จะส่งไปให้ไคลเอนต์ใช้สำหรับเข้ารหัสกุญแจลับ ซึ่งกุญแจสาธารณะนี้จะไม่มีการเข้ารหัสใดๆ เพราะกุญแจสาธารณะมีความปลอดภัยในตัวมันเองอยู่แล้ว

4. ServerHelloDone

จะเป็นข้อความที่บอกให้ไคลเอนต์ทราบว่าเซิร์ฟเวอร์ได้ส่งข้อมูลที่ใช้สำหรับการเริ่มต้นการติดต่อสื่อสารเรียบร้อยแล้วซึ่งข้อความนี้จะไม่มีข้อมูลใด ๆ แต่มันเป็นสิ่งสำคัญต่อไคลเอนต์จะต้องได้รับข้อความนี้ก่อน ไคลเอนต์จึงจะสามารถกระทำเฟสต่อไปของการสร้างการเชื่อมต่อที่ปลอดภัยโดยใช้ SSL โพรโทคอลได้

5. ClientKeyExchange

เมื่อเซิร์ฟเวอร์สิ้นสุดการต่อรองค่าพารามิเตอร์ต่างๆ ที่ใช้ในการสื่อสารด้วย SSL โพรโทคอลแล้ว ไคลเอนต์จะตอบกลับด้วยข้อความ ClientKeyExchange ซึ่งคือการส่งกุญแจลับที่ได้เข้ารหัสด้วยกุญแจสาธารณะที่ได้รับมาจากเซิร์ฟเวอร์

เข้ารหัสถอดรหัสข้อมูลแบบกุญแจเดี่ยว (Symmetric key) จะใช้กุญแจตัวเดียวในการเข้ารหัสถอดรหัสข้อมูล ดังนั้นจึงไม่สามารถส่งกุญแจนั้นผ่านระบบเน็ตเวิร์คได้ SSL โพรโทคอลทำการแก้ปัญหาโดยให้เซิร์ฟเวอร์ส่งกุญแจสาธารณะมาให้แก่ไคลเอนต์ ไคลเอนต์จะทำการสร้างเซสชันคีย์แล้วนำเซสชันคีย์ที่ได้มาเข้ารหัสด้วยกุญแจสาธารณะที่ได้รับมาจากเซิร์ฟเวอร์ เมื่อส่งข้อมูลนี้ผ่านระบบเน็ตเวิร์คจะมีเพียงแค่เซิร์ฟเวอร์ที่ส่งกุญแจสาธารณะมาให้เท่านั้นที่จะสามารถทำการถอดรหัสข้อมูลได้ ซึ่งเป็นการทำให้ไคลเอนต์มั่นใจว่ามีเพียงแค่เซิร์ฟเวอร์ที่เป็นเจ้าของกุญแจสาธารณะเท่านั้นที่จะสามารถถอดรหัสข้อมูลได้ ทำให้เกิดความปลอดภัยขึ้นมาได้ในระดับหนึ่ง

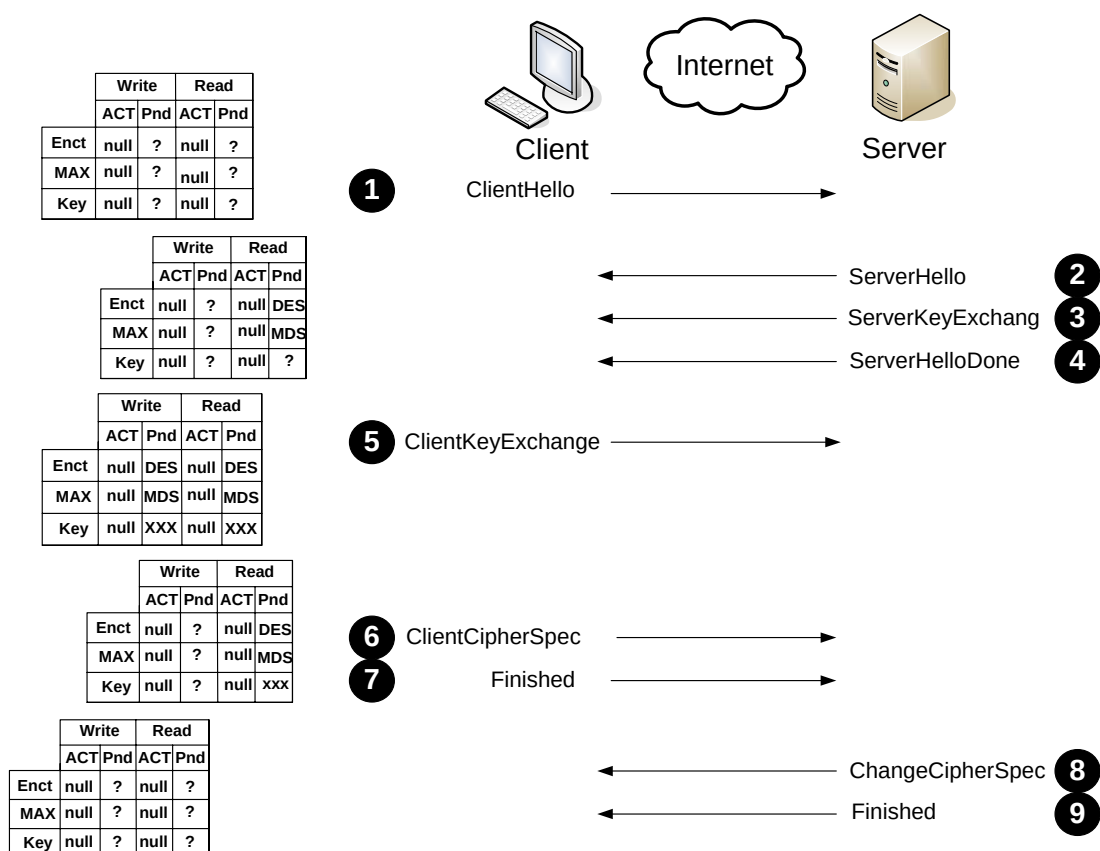
6. ChangeCipherSpec

หลังจากที่ไคลเอนต์ส่งข้อความ ClientKeyExchange เรียบร้อยแล้วจะถือว่าสิ้นสุดการต่อรองค่าพารามิเตอร์ต่าง ๆ ระหว่างไคลเอนต์กับเซิร์ฟเวอร์ ณ จุดนี้ทั้งสองระบบสามารถที่จะสื่อสารกันได้อย่างปลอดภัยโดยใช้ SSL โพรโตคอล ข้อความ ChangeCipherSpec จะเป็นข้อความที่ชี้ให้เห็นอย่างชัดเจนว่าการรับข้อมูลอย่างปลอดภัยโดยใช้ SSL โพรโตคอลสามารถทำงานได้แล้ว

กระบวนการในการรับส่งข้อมูลของ SSL โพรโตคอล ข้อมูลที่ทำการส่งไปจะต้องประกอบไปด้วยชนิดของอัลกอริทึมที่ใช้ในการเข้ารหัส, ข้อมูลที่ใช้ตรวจสอบความถูกต้องของข้อมูล(MIC), กุญแจที่ได้มาจากอัลกอริทึมดังกล่าว SSL โพรโตคอลจะต้องมีการตรวจสอบข้อมูลหลายๆ อย่างโดยเฉพาะอย่างยิ่ง เซสชันคีย์ โดยการตรวจสอบทิศทางระหว่างทิศทางในการรับและการส่งต้องมีความแตกต่างกัน นั่นคือ เซตของคีย์หนึ่งจะเป็นตัวรักษาความปลอดภัยสำหรับไคลเอนต์ในการส่งข้อมูล และเซตของอีกคีย์หนึ่งจะเป็นตัวรักษาความปลอดภัยสำหรับไคลเอนต์ในการรับข้อมูล) ถ้ามีการแยกในความแตกต่างของอัลกอริทึมที่ใช้ทั้งในการรับและการส่งข้อมูลก็จะเป็นวิธีที่ดีแต่ SSL โพรโตคอลไม่มีตัวเลือกนี้อยู่(

ทั้งในฝั่งไคลเอนต์และฝั่งเซิร์ฟเวอร์ SSL โพรโตคอลจะกำหนดให้มีสถานะในการรับข้อมูลเรียกว่า read state และในการส่งข้อมูลเรียกว่า write state จากรูปจะเห็นว่า read และ write state จะถูกแบ่งออกเป็นอีกสองสถานะคือ active และ pending ดังนั้นจะมีทั้งหมดสี่สถานะด้วยกันคือ active write state, pending write state, active read state, pending read state

จากรูปใช้ตัวย่อเป็น Act และ pending จากรูปใช้ตัวย่อเป็น Pnd อัลกอริทึมในการเข้ารหัสถอดรหัสใช้ตัวย่อเป็น Encr ข้อมูลที่ใช้ตรวจสอบความถูกต้องของข้อมูลใช้ตัวย่อเป็น MAC (Message Authentication Code) และคีย์จะใช้เป็น key จากรูป 5-2 และ 5-3 ซึ่งสามารถอธิบายทั้งสี่สถานะได้ดังนี้



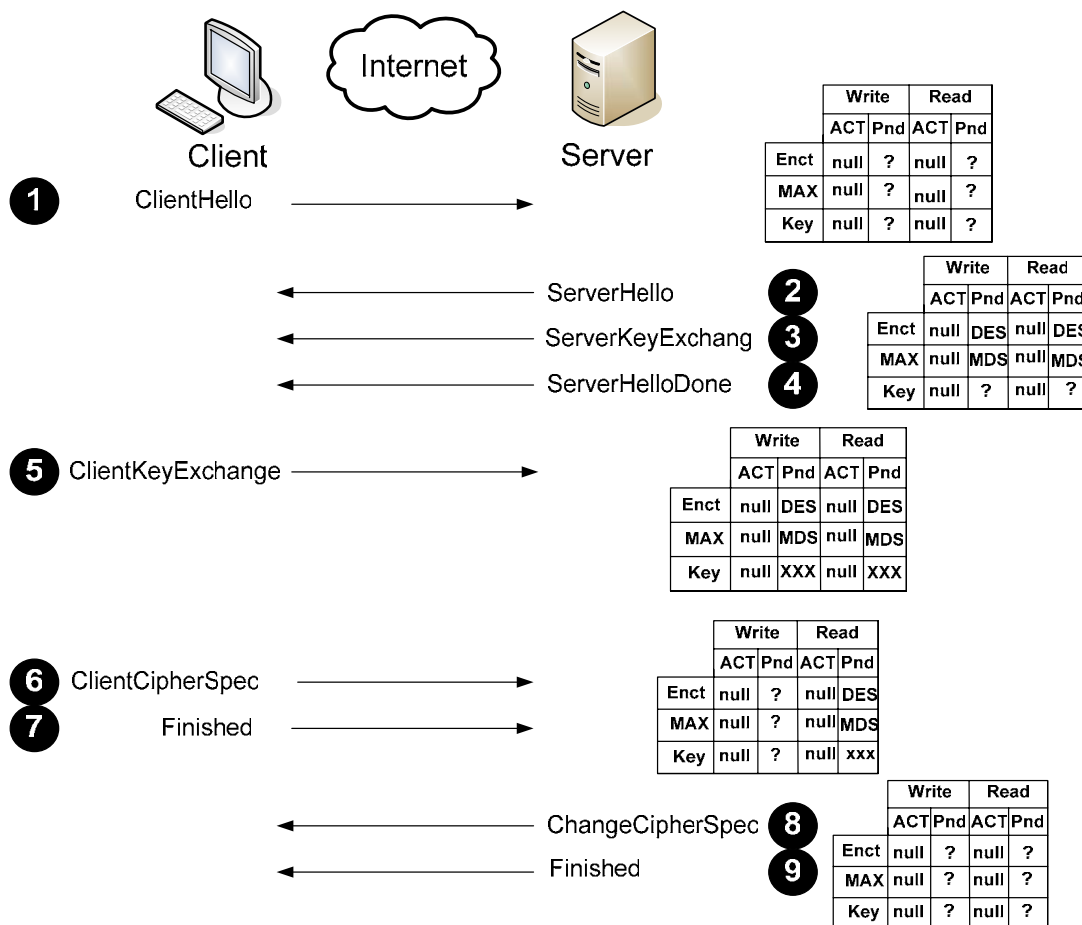
รูปที่ 5-2

แสดงสถานะในการรับส่งข้อมูลของไคลเอนต์ตามกระบวนการของ SSL โพรโทคอล

ฝั่งไคลเอนต์

1. เมื่อไคลเอนต์เริ่มติดต่อสื่อสารด้วย SSL โพรโทคอลโดยการส่งข้อความ ClientHello จะทำการเซตสถานะ active ทั้งสองสถานะเป็น null สถานะ pending จะไม่มีการทำอะไรกับมัน
2. เมื่อไคลเอนต์ได้รับข้อความ ServerHello ในขณะนี้ไคลเอนต์จะทราบว่าเซิร์ฟเวอร์ทำการเลือกอัลกอริทึมใดในการติดต่อเซตขั้นนี้ ไคลเอนต์ทำการปรับข้อมูลของอัลกอริทึมในการเข้ารหัส ถอดรหัส และ ข้อมูลที่ใช้ตรวจสอบความถูกต้องของข้อมูลในสถานะ pending ทั้งสอง จากข้อมูลที่ได้รับมาดังรูป
3. เมื่อไคลเอนต์ได้สร้างเซตขั้นคีย์และส่งข้อความ ClientKeyExchange เรียบร้อยแล้ว จะทำให้ทราบถึงคีย์ที่จะใช้ในการติดต่อสื่อสาร จึงทำการปรับข้อมูลของคีย์ในสถานะ pending ทั้งสอง ดังรูป

4. เมื่อไคลเอนต์ส่งข้อความ ChangeCipherSpec ที่สถานะ write จะเปลี่ยนจากสถานะ pending ไปเป็นสถานะ active พร้อมกับปรับข้อมูลจากข้อมูลของสถานะ pending และทำการรีเซ็ตค่าสถานะ pending ให้ไม่มีค่าใด ๆ ณ จุดนี้ไคลเอนต์จะสามารถทำการส่งข้อมูล (ciphertext) ที่ถูกเข้ารหัสด้วยอัลกอริทึม DES และ ค่าตรวจสอบความถูกต้องที่ใช้ อัลกอริทึม MD5 ได้



รูปที่ 5-3

แสดงสถานะในการรับส่งข้อมูลของเซิร์ฟเวอร์ตามขั้นตอนการของ SSL โพรโทคอล

ฝั่งเซิร์ฟเวอร์

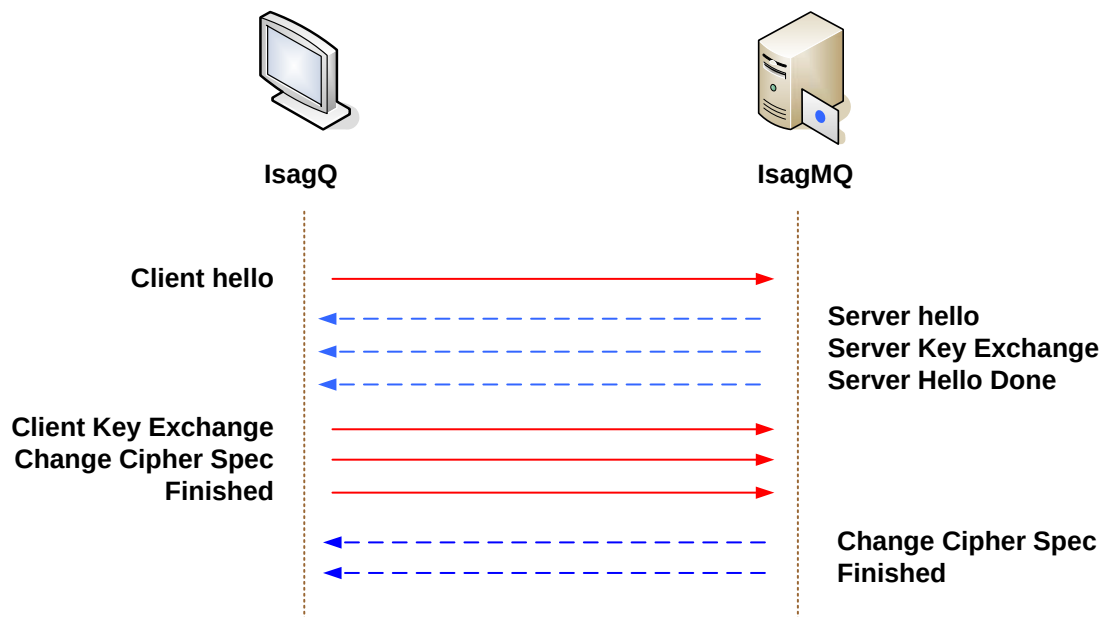
1. เมื่อเซิร์ฟเวอร์ได้รับข้อความ ClientHello มันจะทำการเซตค่าสถานะ active ของทั้งสองให้เป็น null และสถานะ pending จะไม่มีการทำอะไรกับมัน
2. เมื่อเซิร์ฟเวอร์ทำการส่งข้อความ ServerHello ณ จุดนี้มันจะทราบถึงอัลกอริทึมที่ใช้สำหรับเซสชันนี้และจะทำการปรับปรุงข้อมูลเหล่านี้ในสถานะของ pending ดังรูป แต่ข้อมูลของคีย์ในสถานะ pending ในตอนนี้จะอยู่ในสถานะไม่ทราบค่า
3. เมื่อเซิร์ฟเวอร์ได้รับข้อความ ClientKeyExchange จะทำให้มันทราบค่าของคีย์ที่จะใช้ในการติดต่อสื่อสารของเซสชันนี้ ดังนั้นมันจะทำการปรับปรุงข้อมูลของคีย์ในสถานะ pending
4. เมื่อเซิร์ฟเวอร์ได้รับข้อความ ChangeCipherSpec ที่สถานะ read มันจะทำการเปลี่ยนจากสถานะ pending ไปเป็นสถานะ active พร้อมกับปรับข้อมูลจากข้อมูลของ สถานะ pending และทำการรีเซตค่าสถานะ pending ให้ไม่มีค่าใดๆ ณ จุดนี้เซิร์ฟเวอร์สามารถที่จะรับข้อมูล (ciphertext) ที่ถูกเข้ารหัสด้วยอัลกอริทึม DES และค่าตรวจสอบความถูกต้องของข้อมูลที่ใช้อัลกอริทึม MD5 ได้

7. Finished

ทันทีหลังจากที่ได้ทำการส่งข้อความ ChangeCipherSpec แต่ละระบบจะทำการส่งข้อความ Finished ซึ่งเป็นข้อความที่ใช้การตรวจสอบความถูกต้องของข้อมูลที่ได้ทำการส่งไป เช่น ข้อมูลเกี่ยวกับคีย์ รายละเอียดเกี่ยวกับการต่อรองค่าพารามิเตอร์ต่างๆ ในครั้งก่อน และข้อมูลใดๆ ที่จะเป็นการตัวระบุตัวตนของทั้งเซิร์ฟเวอร์และไคลเอนต์ โดยค่าเหล่านี้จะต้องทำให้เป็นแฮชแวลู (hash value) ก่อนทำการส่งออกไป

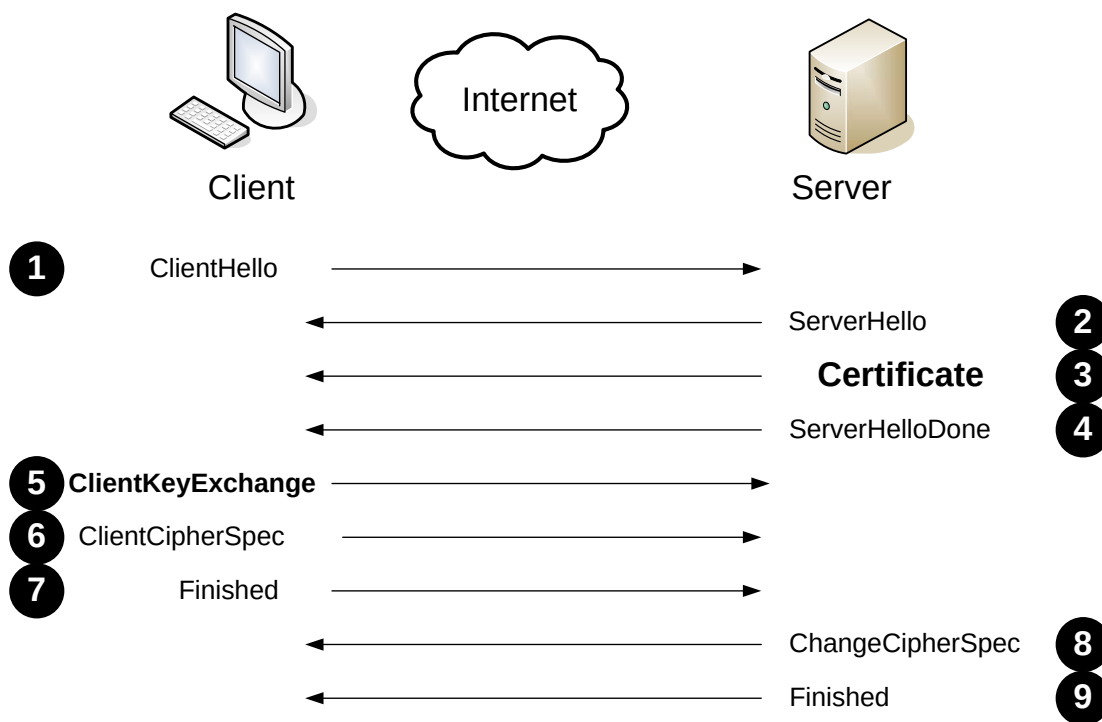
1.4 การสร้างการเชื่อมต่อแบบ SSL โพรโทคอลโดยใช้การเข้ารหัสข้อมูลและมีการพิสูจน์ตนของเซิร์ฟเวอร์

กระบวนการการทำงานของ SSL โพรโทคอลในข้างต้นที่ใช้การเข้ารหัสข้อมูลเพียงอย่างเดียวระหว่างสองระบบนั้นมันยังไม่มีความปลอดภัยเพียงพอ พิจารณาได้จากรูปที่ 5-4 เมื่อ Alice ที่มีบทบาทเป็นไคลเอนต์และ Bob ที่มีบทบาทเป็นเซิร์ฟเวอร์ และมีบุคคลที่อยู่ตรงกลางชื่อว่า Trudy โดยขั้นแรก Trudy จะแสดงบทบาทตัวเองเป็น Alice โดยบอก Bob ว่า “I’m Alice” และ Bob จะส่งข้อความว่า “I’m Bob” พร้อมส่งกุญแจสาธารณะของตนเองให้แก่ Trudy ซึ่งในขณะนี้ได้ปลอมตัวเป็น Alice เมื่อได้กุญแจสาธารณะของ Bob Trudy จะทำการสร้างกุญแจลับขึ้นมาแล้วใช้กุญแจสาธารณะของ Bob มาเข้ารหัสกุญแจลับที่ได้สร้างขึ้นมาเสร็จแล้วก็ทำการส่งข้อมูล (Cipher text) นั้นให้แก่ Bob ณ จุดนี้ Trudy สามารถที่จะติดต่อสื่อสารกับ Bob ได้ จากนั้น Trudy จะทำการปลอมตัวเองให้เป็น Bob ที่มีบทบาทเป็นเซิร์ฟเวอร์ โดย Trudy จะรับข้อความจาก Alice ว่า “I’m Bob” พร้อมส่งกุญแจสาธารณะของตนเองให้แก่ Alice ณ ตอนนี Trudy ก็สามารถติดต่อสื่อสารกับ Alice ได้ เมื่อถึงจุดนี้ Trudy จะทราบข้อมูลทุกอย่างที่ Bob กับ Alice ติดต่อกัน ดังนั้น Trudy สามารถสร้างความเสียหายให้แก่ Bob และ Alice ได้ ซึ่งเราเรียกว่าปัญหาที่เกิดขึ้นนี้ว่า การโจมตีแบบแมนอินเดอะมิดเดิล (Man-in-the-middle)



รูปที่ 5-4 แสดงการโจมตีแบบแมนอินเดอะมิดเดิล (Man-in-the-middle)

โพรโตคอลได้ทำการแก้ไขปัญหาดังกล่าวโดยได้เพิ่มวิธีการที่เรียกว่าการพิสูจน์ตัวตนของเซิร์ฟเวอร์จากรูปที่ 5-5 จะเห็นได้ว่าข้อความ Certificate ถูกนำมาแทนที่ข้อความ ServerKeyExchange



รูปที่ 5-5 แสดงสองข้อความใหม่ที่ใช้สำหรับการพิสูจน์ตัวตนของเซิร์ฟเวอร์

1.5 ใบรับรองสิทธิ์ (Certificate)

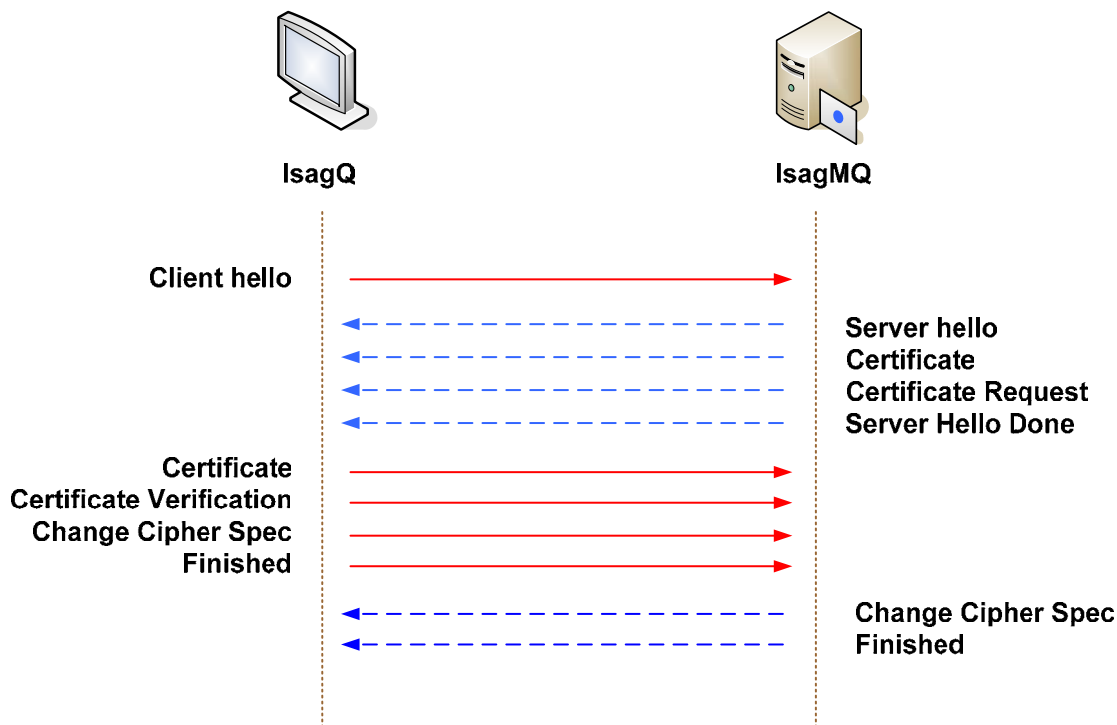
การที่ไคลเอ็นต์จะสามารถทำการระบุตัวตนของเซิร์ฟเวอร์ได้ เซิร์ฟเวอร์จะส่งข้อมูลต่างๆ ที่สามารถเป็นการระบุตัวตนได้เช่น ชื่อ, ฤๅญแจสาธารณะของเซิร์ฟเวอร์ ไปให้แก่ CA (Certificate Authority) ซึ่ง CA จะแบ่งออกเป็นสองลักษณะคือ Internal CA และ External CA (ซึ่งจะได้กล่าวในหัวข้อที่ 3 Certificate Authority(CA)) และ CA จะทำการกระบวนการที่เรียกว่า Sign() โดยจะใช้คีย์ส่วนตัวของ CA ทำการ Sign ข้อมูลที่ได้รับมาจากเซิร์ฟเวอร์ และเมื่อนำฤๅญแจสาธารณะของ CA กับข้อมูลที่ได้มาจากการ Sing ดังกล่าวจาก CA แล้วมาทำการกระบวนการที่เรียกว่าการ Verify() ก็จะสามารถพิสูจน์ตัวตนของเซิร์ฟเวอร์ได้ จากรูปไคลเอ็นต์จะมีฤๅญแจสาธารณะของ CA ที่เป็น CA ที่เซิร์ฟเวอร์ได้นำข้อมูลต่างๆ ไปให้ CA นั้นทำการ Sign ให้ โดยข้อความ Certificate ของฝั่งเซิร์ฟเวอร์จะประกอบไปด้วย ฤๅญแจสาธารณะของเซิร์ฟเวอร์และข้อมูลได้รับการ Sing มาจาก CA นั้น ส่งไปให้แก่ไคลเอ็นต์

1.6 ClientKeyExchange

เมื่อไคลเอนต์ได้รับข้อความ Certificate จากเซิร์ฟเวอร์ จะใช้ฟังก์ชัน Verify() ในการพิสูจน์ตัวตนของเซิร์ฟเวอร์ ถ้าพิสูจน์ตัวตนถูกต้อง ไคลเอนต์จะนำกุญแจสาธารณะของเซิร์ฟเวอร์ที่ได้ส่งมาด้วยนั้น นำมาเข้ารหัสกุญแจลับที่ได้สร้างขึ้นตามกระบวนการเดิม

1.7 การแยกระหว่างการพิสูจน์ตนกับการเข้ารหัสข้อมูล

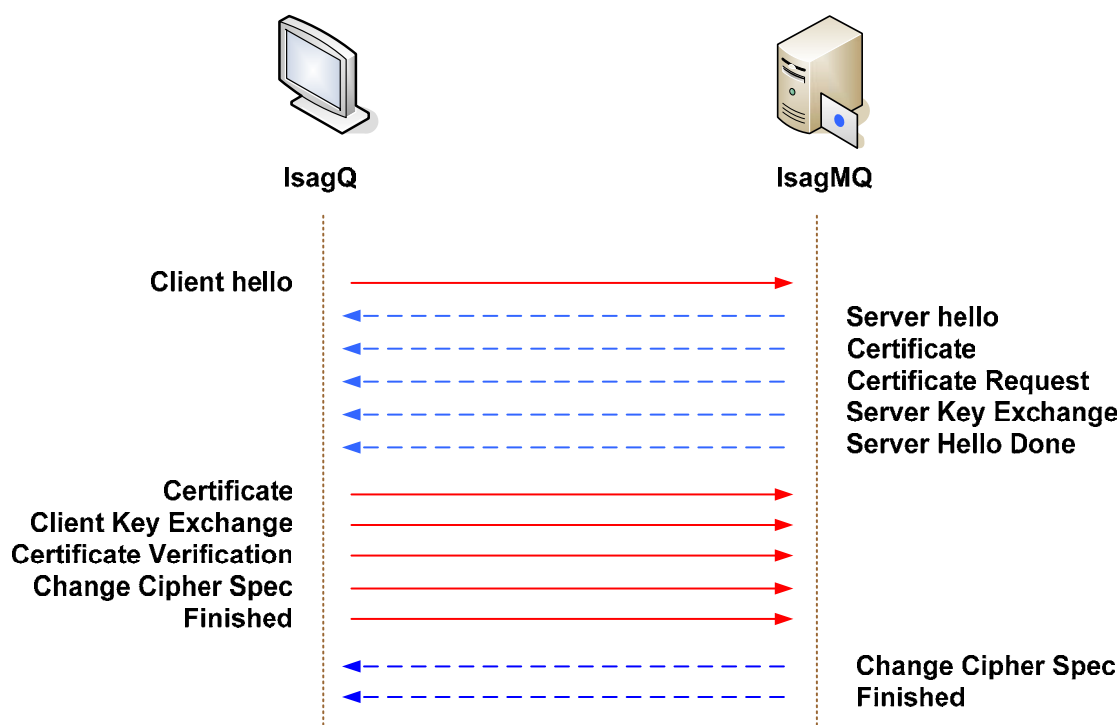
การใช้ข้อความ Certificate เพียงอย่างเดียวในการทำทั้งการพิสูจน์และการเข้ารหัสข้อมูลนั้น เป็นวิธีการที่ไม่ดีนัก เนื่องจากว่า ยกตัวอย่างเช่นมีหลาย ๆ อัลกอริทึมที่ใช้ในการสร้างกุญแจสาธารณะที่มีไว้ใช้สำหรับการ Sign ข้อมูลเท่านั้น ไม่สามารถนำมาใช้ในการเข้ารหัสข้อมูลได้ เช่น DSA (Digital Signature Algorithm) ดังนั้นจึงมีการแยกข้อความระหว่างพิสูจน์ตนกับข้อความสำหรับการเข้ารหัสออกจากกัน ดังรูปที่ 5-6 เซิร์ฟเวอร์จะให้ข้อความ Certificate สำหรับการพิสูจน์ตนของเซิร์ฟเวอร์ ส่งไปให้แก่ไคลเอนต์ก่อนต่อจากนั้นจึงทำการส่งข้อความ ServerKeyExchange ที่ใช้สำหรับเข้ารหัสข้อมูลให้แก่ไคลเอนต์ ในฝั่งของไคลเอนต์ถ้าการพิสูจน์ตัวตนของเซิร์ฟเวอร์ถูกต้อง ไคลเอนต์จะใช้กุญแจสาธารณะของเซิร์ฟเวอร์ทำการเข้ารหัสกุญแจลับที่ได้สร้างขึ้นตามกระบวนการของ SSL โพรโทคอล



รูปที่ 5-6 การใช้ระบบพิสูจน์ตนอย่างเดียว

1.8 การพิสูจน์ตัวตนของไคลเอนต์

เมื่อเซิร์ฟเวอร์มีความต้องการที่จะพิสูจน์ตัวตนของฝั่งไคลเอนต์ ไคลเอนต์จะต้องนำข้อมูลต่างๆ ของไคลเอนต์ไปให้แก่ CA ที่เซิร์ฟเวอร์เชื่อถือ ทำการ Sign ข้อมูลให้เพื่อที่ไคลเอนต์จะสามารถพิสูจน์ตัวตนได้ ดังนั้นไคลเอนต์จะต้องมีการสร้างกุญแจสาธารณะของตนเองขึ้นมาเพื่อใช้ในการพิสูจน์ตัวตนของไคลเอนต์ โดยกุญแจสาธารณะของไคลเอนต์นี้จะใช้ในการพิสูจน์ตัวตนเท่านั้นไม่ได้ใช้ในการเข้ารหัสข้อมูลใด ๆ จากรูปที่ 5.7 ข้อความที่เพิ่มขึ้นมาคือ



รูปที่ 5-7 แสดงการพิสูจน์ตนและเข้ารหัสของฝั่งไคลเอนต์

Certificate Request

จะเป็นข้อความจากเซิร์ฟเวอร์ที่เป็นการบอกให้แก่ไคลเอนต์ทราบว่าเซิร์ฟเวอร์มีความต้องการที่จะทำการพิสูจน์ตัวตนของไคลเอนต์ โดยจะเห็นได้ว่าข้อความนี้อยู่หลังข้อความ Certificate เนื่องจากจะต้องมีการพิสูจน์ตัวตนของเซิร์ฟเวอร์ให้ถูกต้องก่อน เมื่อการพิสูจน์ตัวตนของเซิร์ฟเวอร์ถูกต้อง เซิร์ฟเวอร์จึงทำการร้องขอการพิสูจน์ตัวตนของฝั่งไคลเอนต์

Certificate

เป็นข้อความของฝั่งไคลเอนต์ที่เป็นข้อมูลที่ไคลเอนต์ได้รับจากการส่งข้อมูลของไคลเอนต์ เช่น ฤกษ์แจสารณะของไคลเอนต์ ไปให้แก่ CA ที่เซิร์ฟเวอร์เชื่อถือทำการ Sign ข้อมูลดังกล่าว

Certificate Verify

เป็นข้อความที่คล้าย ๆ กับข้อความ Finished แต่สิ่งที่แตกต่างกันคือ ข้อมูลของคีย์ซึ่งถ้าเป็นข้อความ Finished จะเป็นข้อมูลของฤกษ์แจลับ แต่ถ้าเป็นข้อความ CertificateVerify จะเป็นข้อมูลของฤกษ์แจสารณะที่ไคลเอนต์ใช้ในการพิสูจน์ตัวตน

2 OpenSSL

2.1 OpenSSL คืออะไร

Open SSL คือโปรเจกต์ของทีมงานหนึ่งที่พยายามในการพัฒนา OpenSSL ที่สามารถไปเรียกใช้การทำงานของ Secure Socket Layer (SSL) และ Transport Layer Security (TLS) โพรโตคอลได้อย่างมีประสิทธิภาพและทั้งนี้ Open Source นี้จะประกอบด้วยไลบรารีต่าง ๆ ที่เกี่ยวข้องกับการเข้ารหัสและถอดรหัสข้อมูลโดยใช้อัลกอริทึมต่าง ๆ ที่มีความปลอดภัยสูงเช่น RSA, DSA, 3DES ฯลฯ ไว้อย่างครบถ้วน ซึ่งข้อมูลและรายละเอียดต่าง ๆ หาได้ที่ www.openssl.org

2.2 การใช้งาน OpenSSL

ก่อนที่จะใช้งาน OpenSSL เราจะต้องทำการติดตั้ง OpenSSL เสียก่อนถึงจะมี ไลบรารีและคำสั่งในการสร้างคีย์ให้ใช้งาน เริ่มต้นให้ดาวน์โหลด Source OpenSSL 0.9.7e (25-oct-2004 including important bugfixes) จาก www.openssl.org ที่เลือกเวอร์ชันนี้เพราะมีเสถียรภาพและปลอดภัยแล้ว วิธีการติดตั้งดูได้จาก Readme.txt ที่อยู่ในซอร์สหรือถ้าหากได้ทำการติดตั้ง Apache-ssl ไว้แล้วก็ไม่จำเป็นต้องติดตั้ง Source OpenSSL อีกเพราะมีไลบรารีจาก Apache-ssl แล้วจากนั้นก็สามาริใช้คำสั่งและไลบรารีต่าง ๆ ของ OpenSSL ได้ และที่สำคัญในขณะที่กำลังติดตั้งเราต้องสังเกตดูว่าเก็บเฮเดอร์ไฟล์และไลบรารีเก็บอยู่ที่ไหนด้วยเพราะต้องใช้ตอนคอมไพล์โปรแกรม

2.2.1 ตำแหน่งเฮเดอร์ไฟล์และไลบรารี

การที่เราจะทำการคอมไพล์ โปรแกรมที่ Include เฮเดอร์ไฟล์ของ Openssl เราต้องรู้จักตำแหน่งที่เก็บเฮเดอร์ไฟล์และไลบรารีเสียก่อน ถ้าเราทำการติดตั้ง OpenSSL แบบมาตรฐานแล้วเฮเดอร์ไฟล์และไลบรารีของ OpenSSL ก็จะอยู่ที่เฮเดอร์ไฟล์และไลบรารีมาตรฐานของ ลินุกซ์ คือ

-/usr/include/openssl	สำหรับเฮเดอร์ไฟล์
-/usr/lib/	สำหรับไลบรารี
-ไลบรารีที่จำเป็นคือ libssl.a และ libcrypto.a	

ถ้าหากหาเฮเดอร์ไฟล์และไลบรารี ที่ตำแหน่งมาตรฐานไม่เจอหลังจากที่ได้ทำการติดตั้งไปแล้วให้ลองไปดูที่ /usr/local/ssl/ ซึ่งอาจจะเจอหรืออยู่ตามตำแหน่งที่กล่าวไว้ตอนติดตั้ง OpenSSL

2.2.2 การคอมไพล์

ถ้าหากเซิร์ฟเวอร์ไฟล์และไลบรารีของ OpenSSL อยู่ตำแหน่งมาตรฐานของเซิร์ฟเวอร์ไฟล์และไลบรารีของ ลินุกซ์ เราไม่ต้องอ้างถึง path เลย ทำการคอมไพล์และลิงก์ไลบรารีได้ดังนี้

```
gcc -c TestOpenssl.c                                /*Compile*/
gcc -o TestOpenssl TestOpenssl.o -lssl -lcrypto      /*Link Library*/
```

ถ้าหากเซิร์ฟเวอร์ไฟล์และไลบรารีของ OpenSSL ไม่ได้อยู่ตำแหน่งมาตรฐานของเซิร์ฟเวอร์ไฟล์และไลบรารีของลินุกซ์ เราต้องอ้างถึง path ก่อนจะทำการคอมไพล์และลิงก์ไลบรารี ได้ดังนี้

```
gcc -c TestOpenssl.c -L/~path header file~          /*Compile*/
gcc -o TestOpenssl TestOpenssl.o -L/~path library file -lssl -lcrypto /*Link Library*/
```

3. Certificate Authority(CA)

CA เป็นองค์กรหรือสมาคมที่จะออกใบรับรองสิทธิให้แก่ผู้ขอ ใบรับรองสิทธิก็คือ ใบที่ผู้ใช้พิสูจน์ถึงตนเองเมื่อต้องการติดต่อกับผู้อื่น เหมือนการใช้บัตรประชาชน ต่างกันตรงที่ใบรับรองสิทธินี้เอาไว้รับรองสิทธิในการสื่อสารบนระบบ ผู้ขอใบรับรองสิทธิจากCA จะต้องมั่นใจในใบรับรองสิทธินั้นแต่ไม่ได้หมายความว่าCAนั้นจะไม่มี ความผิดพลาดเลย ตัวอย่างองค์กรที่เป็นCA เช่น Verisign รายละเอียดต่างๆ หาได้ที่ www.verisign.com

ประเภทของ CA มี 2 ชนิด

- Public CAs หรือ External CAs

เป็นองค์กรที่ออกใบรับรองสิทธิ์ให้แก่บุคคลหรือองค์กรโดยทั่วไปเช่น Verisign จะออกใบรับรองสิทธิ์ให้กับเว็บไซต์ (web sites) หรือแอปพลิเคชันที่ต้องการให้มีการเข้ารหัสและการรับรองสิทธิ์ของผู้ใช้ในการทำกิจกรรมต่าง ๆ เช่น อีคอมเมิร์ซ (e-commerce) จะต้องมีระบบการรับส่งข้อมูลที่ปลอดภัยเช่น รหัสผ่านของบัตรเครดิตของลูกค้า ซึ่งโดยมากแล้วถ้าเป็น External CAs ถ้าเราไปขอใบรับรองสิทธิ์เราจะไม่ได้อะไร ๆ จะต้องมีการจ่ายเงินให้แก่ External CAs นั้นๆ ก่อน

- Private CAs หรือ Internal CAs

เป็นองค์กรที่ออกใบรับรองสิทธิ์ให้แก่คนที่อยู่ในองค์กรเพื่อใช้งานภายในองค์กรเอง องค์กรภายนอกไม่สามารถใช้ CA และจะไม่สามารถไว้วางใจการออก CA แบบนี้ได้แต่ขึ้นอยู่กับ Internal CAs นั้น ๆ ว่ามีกฎระเบียบเป็นอย่างไร ซึ่ง CA ประเภทนี้จะออกใบรับรองสิทธิ์ให้ฟรีเพราะถือว่าบุคคลเหล่านั้นอยู่ภายในองค์กร

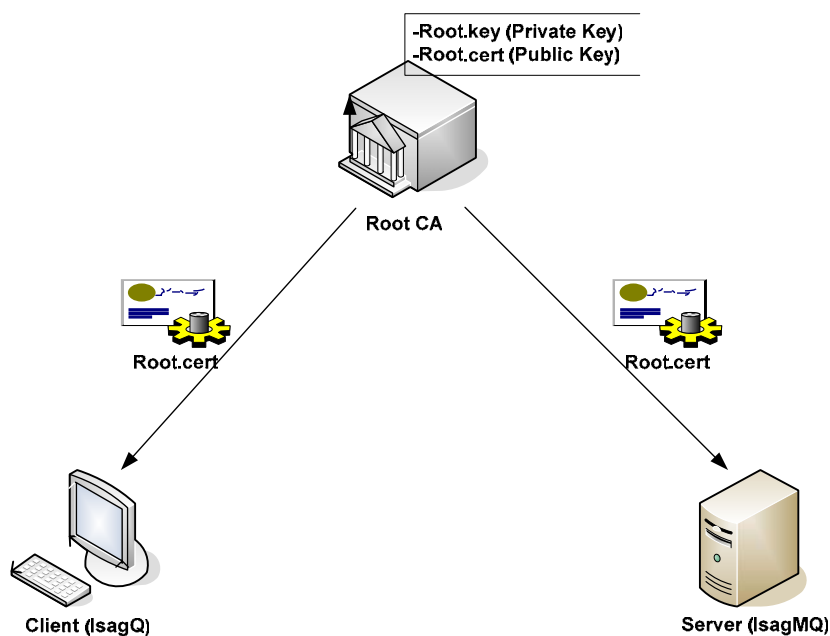
การทำโครงการนี้ได้ทำระบบ CA แบบ Private CA เพราะเห็นว่าเหมาะสมที่จะออกใบรับรองสิทธิ์ให้แก่ผู้ใช้ของ IsagMQ เท่านั้น เพื่อการสื่อสารอย่างปลอดภัยภายในกลุ่ม ในที่นี้จะอธิบายการสร้างและการออกแบบ Private CA เท่านั้นซึ่งจะอยู่ในส่วนการออกแบบและพัฒนา

3.1 Third Party

- Root CA เป็นผู้ที่ออกใบรับรองสิทธิ์ให้แก่ผู้ร้องขอ
- Client เป็นผู้ขอใบรับรองสิทธิ์เพื่อใช้ในการพิสูจน์ตน
- Server เป็นตรวจสอบใบรับรองสิทธิ์เพื่อใช้ในการพิสูจน์ตน

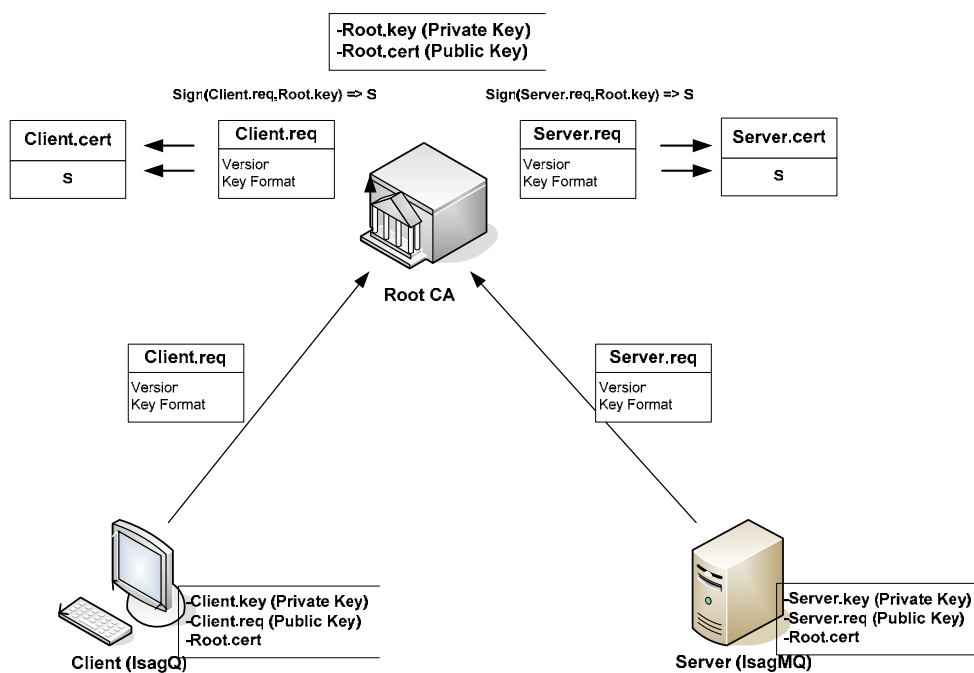
3.2 ขั้นตอนในการร้องขอใบรับรองสิทธิ์ จาก Public CA

1. ผู้ร้องขอซึ่งในที่นี้จะประกอบด้วย Client และ Server จำเป็นต้องมี ใบรับรองสิทธิ์ของ Root เพื่อเก็บไว้ใช้ในระบบพิสูจน์ตน ดังรูป 5-8



รูปที่ 5-8 ผู้ร้องขอจำเป็นต้องมี ใบรับรองสิทธิ์ของผู้ออก

2. เมื่อผู้ร้องขอมีใบรับรองสิทธิ์ของ Root CA แล้ว ก็ให้ทำการร้องขอไปยัง Root CA ดังรูป



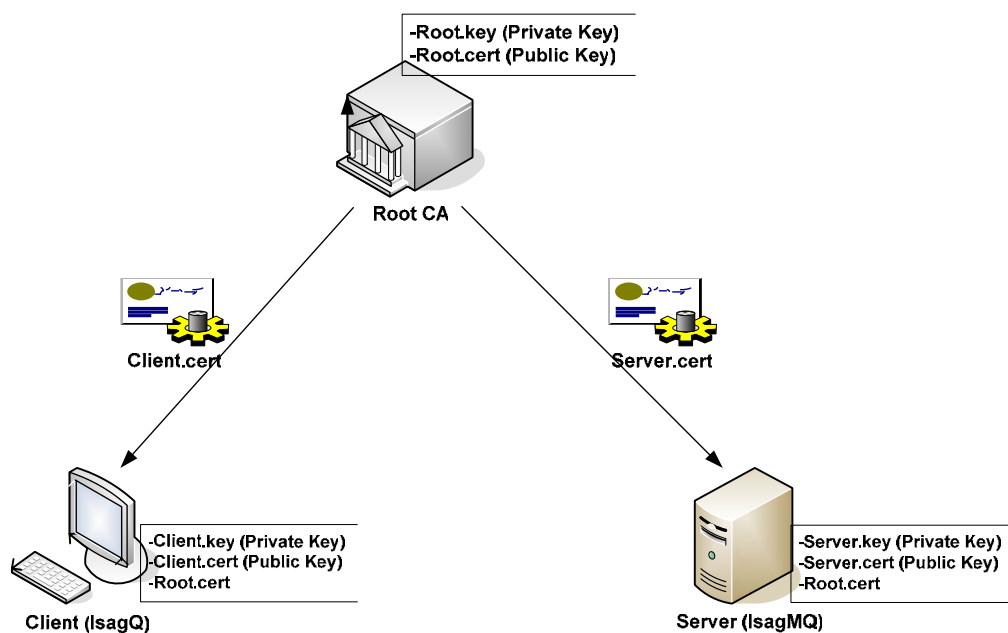
รูปที่ 5-9 ผู้ร้องขอส่งคำร้องไปยังผู้ออกเพื่อให้ผู้ออกทำการ sign ใบรับรองสิทธิ์

จากรูป 5-8 จะเห็นได้ว่า ก่อนที่ผู้ร้องขอจะส่ง Client.req หรือ Server.req ออกไป ผู้ร้องขอจำเป็นต้องมี ไฟล์ 2 ชนิดก่อน คือ Client.key หรือ Server.key ซึ่งคือกุญแจส่วนตัวของผู้ร้องขอ (Private Key) และ Client.req หรือ Server.req ซึ่งคือไฟล์ที่ผู้ร้องขอไปยังผู้ออกใบรับรองสิทธิ์ให้

Client.req หรือ Server.req ที่ส่งไปจำเป็นต้องถูกเข้ารหัส ด้วย Root.cert ก่อนเพื่อที่จะมั่นใจได้ว่า Root CA จะสามารถเปิดดูได้เพียงผู้เดียว

หลังจากที่ Root CA ได้รับ Client.req หรือ Server.req แล้ว ก็จะทำการแฮช (hash) ซึ่งจะได้ค่ามาค่าหนึ่ง แล้วใช้ฟังก์ชัน sign ด้วย Root.key กับ ค่าแฮช ที่ได้ ผลลัพธ์จะได้ค่า S (Digital Signature) ออกมา แล้วนำข้อมูลจาก Client.req หรือ Server.req และค่า S ที่ได้ไปเก็บไว้ใน Client.cert หรือ Server.cert

- ผู้ออกใบรับรองสิทธิ์จะทำการส่ง Client.cert หรือ Server.cert กลับมายัง ผู้ร้องขอ โดยไฟล์ดังกล่าวผู้ร้องขอเท่านั้นที่จะสามารถเปิดดูได้เช่นกัน ดังรูป 5-10

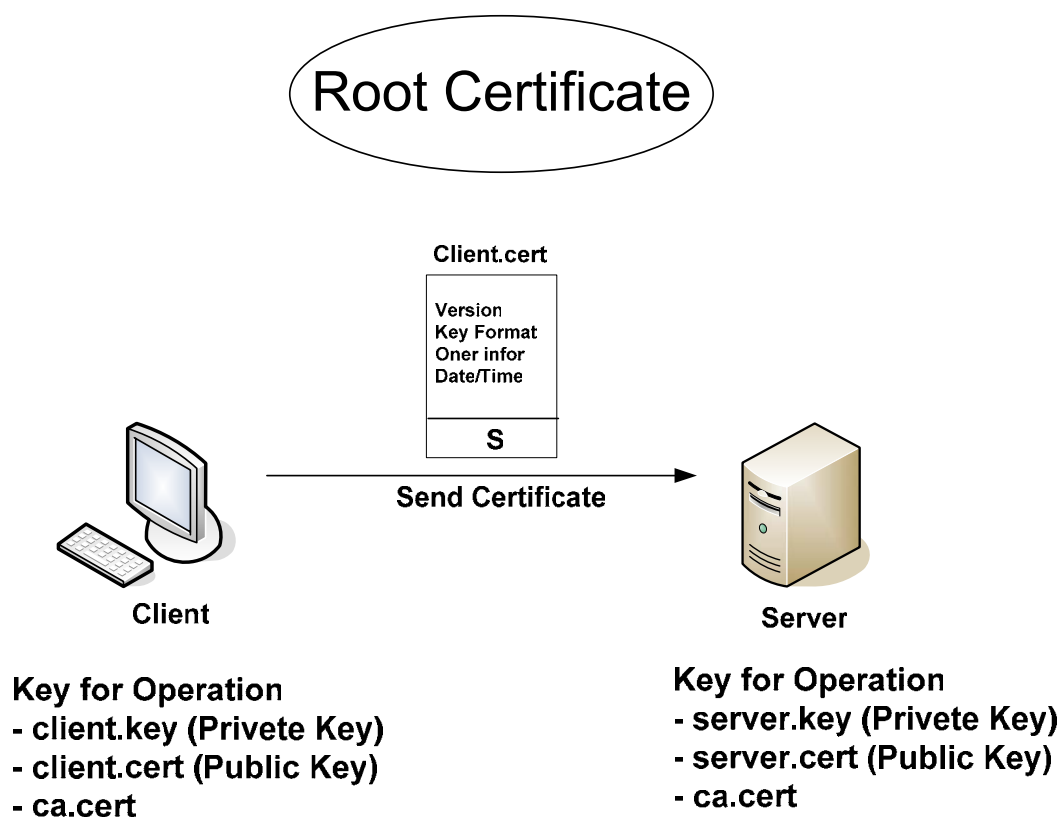


รูปที่ 5-10 ผู้ร้องขอได้รับ ใบรับรองสิทธิ์

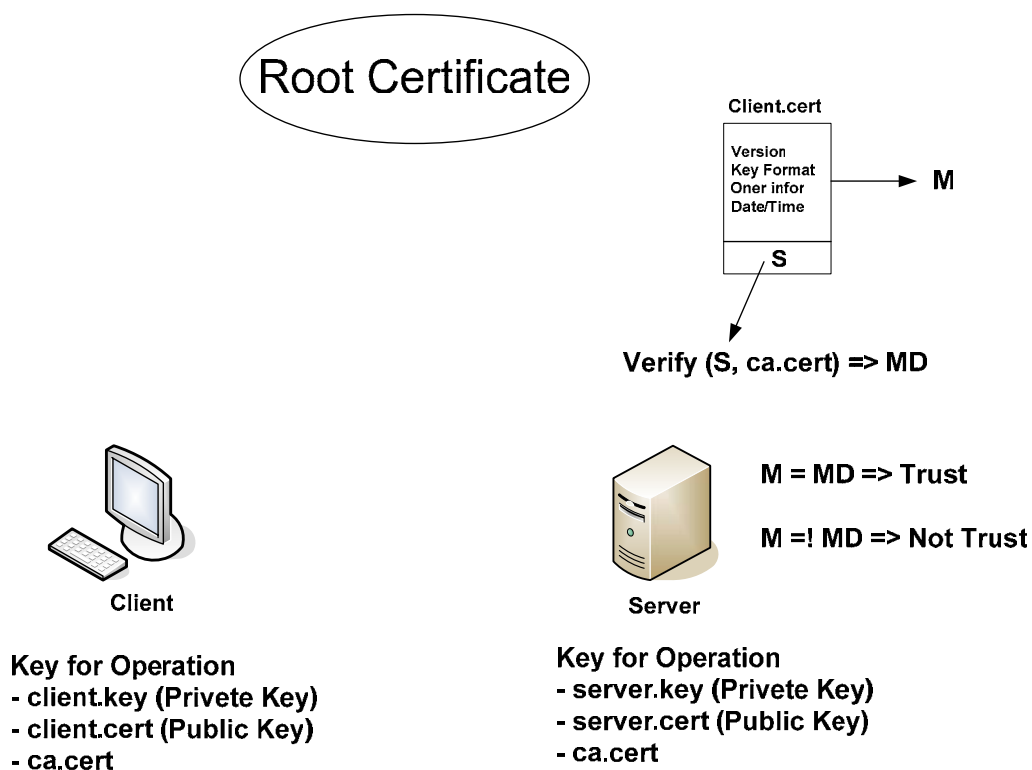
3.3 การสื่อสารกันโดยมีใบรับรองสิทธิ์ (Certificate Authentication)

จากรูป 5-11 เมื่อเซิร์ฟเวอร์ร้องขอการพิสูจน์ตนของฝั่งไคลเอนต์ จากนั้นไคลเอนต์จะส่งข้อความ Certificate (client.cert) ให้แก่เซิร์ฟเวอร์ทำการพิสูจน์ตนของฝั่งไคลเอนต์

จากรูปที่ 5-12 เมื่อเซิร์ฟเวอร์ได้รับไฟล์ client.cert ซึ่งไฟล์นี้ประกอบไปด้วยสองส่วนคือ ส่วนข้อมูลส่วนตัวของไคลเอนต์และค่า S (Digital Signature) กระบวนการพิสูจน์ตนทางฝั่งเซิร์ฟเวอร์คือ จะนำเอาข้อมูลส่วนตัวของไคลเอนต์มาทำ MD5 ได้ค่าแฮช (hash) มาหนึ่งค่าคือค่า M จากนั้นนำค่า S ของไคลเอนต์และกุญแจสาธารณะของ Root Certificate มาเข้าฟังก์ชัน Verify ค่าที่ได้ออกมาคือ MD นำค่า M กับ MD มาเปรียบเทียบกับ ถ้าค่านี้เท่ากันแสดงว่าการพิสูจน์ตัวตนนั้นถูกต้อง ถ้าไม่เท่ากันแสดงว่าการพิสูจน์ตนผิดพลาด



รูปที่ 5-11 แสดงการสื่อสารโดยใช้ใบรับรองสิทธิ์



รูปที่ 5-12 แสดงการพิสูจน์ใบรับรองสิทธิ์โดยฝั่งเซิร์ฟเวอร์